Recherche brute force : swings → TS2Vec → XGBoost

Cet outil collecte les opportunités de retournement sans les filtres d'une stratégie de portefeuille. Les données sont conservées dans une base SQLite. Il est indépendant des moteurs `structure`, `swing-regression` et `swing-patterns`.

Le protocole par défaut est celui demandé :

Étape	Période	Opération
Collecte initiale	2020–2025	Construire les swings, contextes et résultats de tous les trades indépendants
Apprentissage	2020–2023	Entraîner TS2Vec, son scaler et XGBoost
Validation	2024–2025	Choisir la fenêtre, le nombre d'arbres et le seuil d'acceptation
Test séparé	2026	Transformer les nouveaux contextes et prédire avec les modèles et seuils figés

Il n'y a pas de période de préentraînement supplémentaire découpée dans 2020–2023. TS2Vec apprend sur les contextes de cette période entière, sans utiliser de PnL. XGBoost apprend sur ses trades dont le résultat est connu avant le 1er janvier 2024. Un trade de 2023 fermé en 2024 n'est pas un label disponible pour cet apprentissage. De même, un trade de validation fermé en 2026 n'entre pas dans le choix du seuil.

Les embeddings historiques de l'apprentissage sont des reconstructions avec l'encodeur appris sur 2020–2023 : c'est une transformation du jeu d'apprentissage. Les contextes eux-mêmes n'utilisent que les swings connus à l'instant du signal. La validation et le test utilisent ensuite l'encodeur et le scaler figés. Les contextes de début 2024 ou 2026 peuvent naturellement contenir des swings de la fin de la période précédente.

La commande `train` n'évalue pas le test, même si des lignes de 2026 existent déjà dans la base. La commande `test` ne rappelle ni l'apprentissage TS2Vec, ni `fit` XGBoost, ni la sélection d'un seuil.

Règles de collecte

- Un swing high est une bougie haussière bid suivie d'une bougie baissière bid. Son prix est le plus haut bid des deux bougies. Il produit une vente au bid.
- Un swing low est une bougie baissière ask suivie d'une bougie haussière ask. Son prix est le plus bas ask des deux bougies. Il produit un achat à l'ask.
- Le signal devient disponible à la clôture de la seconde bougie. L'ordre au marché s'exécute sur le premier tick strictement ultérieur à son instant d'éligibilité, conformément au modèle d'exécution existant.
- L'ordre non exécuté expire après deux bougies par défaut. **Une position exécutée n'expire pas après deux bougies** : elle reste suivie par son stop et son trailing.
- Le stop initial est le prix du swing, élargi par `--stop-buffer-pips`. Achat : `low ask − marge`. Vente : `high bid + marge`. Le défaut est zéro. Le risque initial et tous les paliers du trailing sont calculés avec

ce stop élargi.
- Le trailing s'active à 1 R par défaut, passe au prix d'entrée, puis avance par paliers de 1 R. Il réutilise le `TrailingStopManager` et le moniteur de stops du bot, sans TP fixe.

Aucun `RiskManager` n'intervient dans la collecte. Il n'y a ni plafond de positions, ni contrôle de capital ou de marge, ni filtre de spread, de distance de stop, de volatilité ou de RR. Chaque opportunité est simulée indépendamment et peut se chevaucher avec toutes les autres. Un stop déjà franchi au fill ne provoque pas un rejet : le trade est exécuté et sa perte immédiate est enregistrée au prix de sortie exécutable.

La valeur `--risk-cash 100` sert seulement à fixer une unité virtuelle comparable pour les résultats : quantité = 100 / distance initiale du stop. Aucun solde n'est débité pour autoriser un autre trade. Le PnL et le PnL en R sont enregistrés. Les sommes de R de ces opportunités ne sont pas un rendement de portefeuille. Si la distance est exactement nulle, le trade et son PnL restent enregistrés, mais son R reste NULL car cette division n'est pas définie.

Les positions encore ouvertes à la fin des données restent sans PnL réalisé. Les ordres expirés n'ont pas de label de trade. Les ticks manquants ne sont pas remplacés par un parcours OHLC synthétique. Les paires de bougies séparées par une interruption ne deviennent pas artificiellement un retournement de deux bougies.

La série de swings

À chaque nouvelle confirmation, on fusionne uniquement les highs bid et les lows ask. La séquence alterne strictement high/low. **Deux swings consécutifs de même nature sont remplacés par le plus récent, même s'il est moins extrême.** Une capture enregistrée lors d'un signal précédent n'est jamais réécrite rétrospectivement.

Les deux modes de contexte sont :

- `duration` : les swings confirmés dans les dernières 12, 24 ou 48 heures par défaut ; la fenêtre est mesurée en temps écoulé et peut donc contenir un nombre variable de points ;
- `pairs` : exactement 4, 8 ou 16 couples, donc respectivement 8, 16 ou 32 swings alternés.

Une fenêtre insuffisamment alimentée ne fournit pas encore d'embedding, mais son trade est tout de même collecté. Si le bid confirme un high et l'ask un low à la même clôture, les deux opportunités sont enregistrées et simulées. Le contexte ambigu est réinitialisé : le moteur n'invente pas leur ordre dans une séquence alternée.

Chaque point contient six valeurs : prix logarithmique relatif au premier swing de la fenêtre, variation logarithmique depuis le swing précédent, nature high/low, délai entre les extrêmes, délai entre les confirmations et délai extrême→confirmation. Les temps sont exprimés en périodes de bougie puis transformés par `log1p`. Les dates d'extrême correspondent aux bougies où ils ont été atteints ; leur disponibilité correspond à la clôture de la seconde bougie.

TS2Vec est le code du sous-module existant, avec six canaux d'entrée. Le scaler est appris uniquement sur l'apprentissage. Les fenêtres de durée peuvent avoir des longueurs différentes ; à l'inférence, seules des séquences de même longueur sont mises dans un batch, sans padding dans l'agrégation max. Les dimensions sont 64 par défaut. Tous les contextes d'apprentissage sont admis par défaut (`--ts2vec-max-train-samples 0`). Pour limiter volontairement le volume de calcul, cette option accepte un plafond. La longueur d'entraînement est limitée à 256 points par fenêtre, configurable avec `--ts2vec-max-train-length` ; l'inférence utilise les fenêtres complètes.

Stockage SQLite

Défaut : `C:\TradingBotData\swing_ts2vec\research.sqlite3`.

Table	Contenu
`swing_runs`	Configuration de collecte, symboles, périodes, fichiers sources, provenance et statut
`swing_events`	Chaque retournement, prix du swing, stop élargi, dates d'ordre/fill/close, statut, PnL et R
`swing_contexts`	Capture immuable de la séquence alternée, fenêtre et six canaux d'entrée
`swing_embedding_models`	Version, scaler, période d'apprentissage et empreinte de TS2Vec
`swing_embeddings`	Vecteur float32 rattaché au contexte et au modèle TS2Vec
`swing_experiments`	Expérience d'apprentissage/validation et modèle choisi
`swing_assignments`	Affectation des contextes à apprentissage, validation ou test et disponibilité des labels
`swing_resolvers`	Métadonnées et résultats des modèles XGBoost
`swing_predictions`	Prédictions, décisions et comparaison sans embedding, rattachées aux contextes
`swing_tests`	Évaluations séparées sur le test avec modèle figé

Les snapshots et clôtures sont validés en transaction à chaque fichier journalier, avec WAL et `synchronous=FULL`. Les poids TS2Vec et XGBoost sont conservés dans des fichiers de modèle, dont les chemins et empreintes sont enregistrés en base. JSON, CSV et Markdown sont des exports de consultation ; la base reste le stockage principal.

Une nouvelle collecte crée un nouveau `run_id` dans la même base. Changer la marge de stop ou le trailing implique de recalculer les résultats dans un nouveau run. L'étape de test vérifie que ces paramètres correspondent à ceux du modèle figé ; ce contrôle de protocole n'est pas un filtre de collecte.

Une interruption conserve les transactions journalières déjà validées et marque le run comme incomplet/échoué. La reprise automatique au milieu du flux n'est pas implémentée : une nouvelle commande `collect` crée un nouveau run. Seuls les runs terminés sont utilisés par l'apprentissage et le test.

Commandes

Depuis `C:\Users\Babs\workspace\TradingBot\ultimateBot` :

```
py -m pip install -r requirements-validator.txt
```

Produire les données 2020–2025, avec une marge d'exemple de 2 pips et les deux modes de contexte :

```
py run_swing_ts2vec.py collect --symbols EURUSD GBPUSD USDCAD USDCHF --start-date
2020-01-01 --end-date 2025-12-31 --candle-interval 60m --durations 12h 24h 48h --
pairs 4 8 16 --stop-buffer-pips 2
```

Exemple:

```
py run_swing_ts2vec.py collect --symbols EURUSD GBPUSD USDCAD USDCHF --start-date
2020-01-01 --end-date 2022-12-31 --candle-interval 10m --pairs 500 --stop-buffer-
pips 10
```

Le résumé affiche le `run_id`. Le copier pour figer la collecte utilisée :

```
$swingTrainRun = "RUN_ID_AFFICHE"
py run_swing_ts2vec.py train --run-id $swingTrainRun --train-start 2020-01-01 --
train-end 2023-12-31 --validation-start 2024-01-01 --validation-end 2025-12-31
```

Ces quatre dates sont déjà les défauts. XGBoost prédit le PnL en R (`--target pnl-r`). Pour prédire la probabilité d'un PnL strictement positif : `--target win`.

Un couple TS2Vec/XGBoost correspond à un intervalle de bougie, un mode de fenêtrage et une taille de fenêtre fixés. Les paires peuvent être regroupées dans cet apprentissage. Par exemple, H1 + durée 24h et H1 + 8 couples produisent deux encodeurs et deux modèles XGBoost distincts. Des encodeurs appris séparément ne partagent pas nécessairement les mêmes coordonnées latentes : leurs embeddings ne sont pas mélangés pour entraîner un XGBoost commun.

Pour fixer une seule fenêtre tout en regroupant toutes les paires du run :

```
py run_swing_ts2vec.py train --run-id $swingTrainRun --candle-interval 60m --
window-mode duration --durations 24h --ts2vec-output-dims 64
py run_swing_ts2vec.py train --run-id $swingTrainRun --candle-interval 10m --
window-mode pairs --pairs 500 --ts2vec-max-train-length 1000 --ts2vec-output-dims
64
```

`--durations 24h` seul sélectionne uniquement cette durée ; `--pairs 8` seul sélectionne uniquement ce nombre de couples. `--pairs 4 8 16` entraîne trois couples de modèles distincts, puis les compare sur la validation. `--window-mode duration` sans liste compare toutes les durées déjà collectées. Sans sélecteur de fenêtre, toutes les variantes collectées sont comparées séparément.

`--symbols` est facultatif : sans cette option, toutes les paires du run participent au même apprentissage pour une fenêtre donnée. Si besoin, `--symbols EURUSD GBPUSD` limite cet ensemble à deux paires ; cela ne crée pas un modèle par paire. Les mêmes sélecteurs s'appliquent aux contextes TS2Vec et aux résultats de trades XGBoost. Le périmètre est enregistré en base et avec les modèles, puis conservé au test.

`--candle-interval` sélectionne une collecte existante à cet intervalle ; il ne reconstruit pas du H4 à partir des swings H1. Il faut d'abord collecter les bougies H4 avec `collect --candle-interval 4h`. Les tailles demandées doivent également avoir été collectées. Un `--run-id` incompatible provoque une erreur explicite ; sans `--run-id`, le programme choisit le dernier run terminé compatible avec les sélecteurs. Les bornes `--train-start`, `--train-end`, `--validation-start` et `--validation-end` ciblent les périodes calendaires et sont indépendantes de la durée du contexte.

La sélection des fenêtres et des seuils se fait uniquement sur la validation et sur les **mêmes opportunités disposant de toutes les fenêtres sélectionnées pour cette comparaison**. Une fenêtre non sélectionnée ne limite pas les effectifs. Un modèle de comparaison sans embedding utilise seulement le sens du trade, le nombre de swings et la durée du contexte. Les résultats et les effectifs sont enregistrés dans la base et dans `rapport.md` / `comparison.csv` du dossier d'expérience affiché. Un résultat de validation négatif reste signalé comme tel : le programme ne le présente pas comme une stratégie gagnante.

Pour collecter ensuite le test, on peut inclure décembre 2025 pour alimenter les premiers contextes de 2026. Garder la même liste de fenêtres, la même marge et le même trailing :

```
py run_swing_ts2vec.py collect --symbols EURUSD GBPUSD USDCAD USDCHF --start-date
2025-12-01 --end-date 2026-12-31 --candle-interval 60m --durations 12h 24h 48h --
pairs 4 8 16 --stop-buffer-pips 2
$swingTestRun = "NOUVEAU_RUN_ID_AFFICHE"
$swingModelDir = "DOSSIER_EXPERIENCE_AFFICHE_PAR_TRAIN"
py run_swing_ts2vec.py test --run-id $swingTestRun --resolver $swingModelDir --
test-start 2026-01-01 --test-end 2026-12-31
```

Seuls les fichiers réellement disponibles sont lus. Si 2026 est incomplet, le rapport de test indique la dernière confirmation observée et les effectifs réellement labellisés ; une période future n'est pas simulée. `test` prend par défaut le modèle choisi par `selected.json`, sans refaire de sélection sur 2026. Les positions non clôturées avant la fin du test ne deviennent pas des pertes artificielles.

`run` combine `collect` puis `train`, sans tester 2026. `--window-mode duration` et `--window-mode pairs` permettent de limiter une collecte à un mode. `--dry-run` affiche les paramètres sans créer de données. Les checkpoints sont enregistrés sous `C:\TradingBotData\swing_ts2vec\experiments\` par défaut ; `--database` et `--output-dir` changent les chemins.

Pour les paires FX classiques, un pip vaut 0,0001 ou 0,01 pour une paire cotée en JPY. `--pip-size` permet de donner explicitement l'unité de prix par pip pour un autre symbole.

Exporter les opportunités ou les embeddings d'un modèle précis :

```
py run_swing_ts2vec.py export --run-id $swingTrainRun --output
C:\TradingBotData\swing_ts2vec\opportunities.csv
py run_swing_ts2vec.py export --run-id $swingTrainRun --model-id MODELE_TS2VEC_ID
--output C:\TradingBotData\swing_ts2vec\embeddings.csv
```

Prédire sur un contexte déjà en base, avec un modèle disponible avant sa date :

```
py run_swing_ts2vec.py score --resolver $swingModelDir --context-id CONTEXT_ID
```

Vérification

Les tests dédiés couvrent les sources bid/ask, le remplacement par le dernier swing, les deux fenêtres, les dates de disponibilité, les stops élargis, les pertes immédiates sans veto, les positions simultanées, l'expiration d'ordre seulement, TS2Vec réel, les deux cibles XGBoost et le test séparé sans appel à `fit`.

Un essai technique sur EURUSD janvier–février 2024 utilise des bornes calendaires raccourcies pour vérifier toute la chaîne : apprentissage en janvier, validation du 1er au 15 février, puis test séparé du 16 au 29 février. Il ne constitue pas l'expérience financière 2020–2026. Ses fichiers sont dans `artifacts/swing_ts2vec_validation/`.

Cet essai lit 3 882 506 ticks et enregistre 520 opportunités : 506 clôturées, 6 encore ouvertes, 7 ordres expirés et 1 en attente à la fin. Quatre variantes de fenêtre sont apprises. Les 111 contextes du test séparé sont prédits après gel du modèle ; 103 disposent d'un résultat connu dans la période du test. Les empreintes des trois checkpoints sont identiques avant et après ce test. Ce petit essai ne démontre aucun avantage financier.

La référence sur tous les ticks et la simulation accélérée produisent exactement les mêmes champs d'événements et les mêmes contextes sur 313 457 ticks et 33 opportunités réelles, avec une marge de 2 pips. Les preuves sont `validation.json` et `tick_equivalence.json` dans ce dossier.

Les 29 tests propres à cet outil et les tests ciblés des stratégies et de l'exécution donnent **234 tests réussis**. Ils vérifient aussi les sélecteurs d'entraînement, le regroupement des paires à fenêtre fixe, l'indépendance vis-à-vis des fenêtres non sélectionnées et le maintien du périmètre au test. L'un d'eux vérifie explicitement les dates par défaut : apprentissage sur les quatre années 2020–2023, validation sur 2024–2025, et aucune lecture de caractéristiques 2026 volontairement rendues inutilisables dans le jeu de test. Une vérification élargie des anciens modules d'embeddings retrouve par ailleurs un échec préexistant du lanceur de sweep : un test attend `adaptive_lookback_max=50` alors que sa configuration actuelle vaut 20. Ces fichiers n'ont pas été modifiés par cet outil.

Les anciennes expériences de mise au point dans `swing_ts2vec_smoke` et `swing_ts2vec_protocol_smoke` ne correspondent pas toutes au protocole final. La version de la politique d'exécution est enregistrée ; les premières collectes qui rejetaient les stops déjà franchis ne sont pas admises pour un nouvel apprentissage avec le collecteur sans veto.